YZM 513 · Proje 1

Yüksek Başarımlı Benzerlik Arama Motoru: arXiv 2025-2026 Özetleri Üzerinde Profilden Paralel Sisteme

Şevval Özlem Sarıtaş · İstanbul Medeniyet Üniversitesi · Güz 2026 · Kod: https://github.com/sevvalsaritas/hpp-for-ai-project

Özet. 19,671 arXiv özetinden (cs.AI / cs.CL / cs.LG, 2025-2026) oluşan bir korpusta top-5 anlamsal benzerlik araması için saf Python bir başlangıç noktasından (100 sorgu x 19,671 doküman: **47.75 s**) başlayıp profil -> darboğaz -> düzeltme döngüsüyle aynı işi **7.48 ms**'de yapan bir sisteme ulaştık (**6,384×**). Hızlanmanın kaynağı sırasıyla: yorumlayıcı yükünü BLAS'a devretmek, normları önceden hesaplamak, BLAS thread sayısını hibrit işlemciye göre ayarlamak ve hızlanma sonrası seri kalan top-k seçimini Numba ile paralelleştirmek (Amdahl). Final sistem 19,671 dokümanda tek sorguyu 303.0 µs'de cevaplıyor, baseline ile 1e-5 toleransta birebir aynı sonucu veriyor ve doküman sayısıyla en fazla doğrusal ölçekleniyor.

1. Giriş

Problem. Kullanıcı bir metin sorgusu girer; sistem korpustaki en benzer 5 makaleyi etkileşimli hızda (hedef: sorgu başına < 10 ms) döndürmelidir. Benzerlik ölçüsü cosine benzerliğidir. Benchmark'a sadece arama (benzerlik + top-k) dahildir; embedding üretimi FAQ gereği hariçtir.

1.1 Veri seti (Seçenek B: arXiv özetleri)

Kaynak: arXiv API (export.arxiv.org), kategoriler cs.AI, cs.CL, cs.LG; Ocak 2025 - Ekim 2026. Tek bir 'en yeniler' sorgusu yalnızca son birkaç haftayı getireceği için (bu kategorilerde ayda ~6-7 bin makale var) veriyi **ay ay**, her aydan eşit sayıda çektik; böylece 22 ayın tamamı temsil edildi. arXiv'in '3 saniyede 1 istek' kuralına uymak için istekler arasında 3,5 s bekledik ve her sayfayı `data/raw/` altında önbelleğe aldık (tekrar çalıştırmada API'ye gidilmez).

Tablo 1: Ön işleme adımları ve her adımdan sonra kalan kayıt sayısı.

Ön işleme adımı	Kalan kayıt
ham kayıt (API)	19,800
tekilleştirme (arXiv id)	19,800
tarih filtresi (2025-2026)	19,800
uzunluk filtresi (≥ 50 kelime)	19,771
sorgu ayırma (100 makale korpustan çıkarıldı)	19,671
karıştırma (seed=42): 'ilk n' alt kümesi tüm dönemi temsil etsin	19,671

Metin temizliğinde satır sonları ve çoklu boşluklar tek boşluğa indirildi, LaTeX komutları ve $ { } karakterleri silindi (matematiğin içeriği korundu). **Sorgular**, korpustan çıkarılan 100 makalenin **başlıklarıdır**: sorgu kendi özetini bulamaz, gerçek bir 'bu konuda benzer makaleler' araması simüle edilir.

Tablo 2: Veri seti istatistikleri.

İstatistik	Değer
Doküman (korpus) / sorgu	19,671 / 100
Özet uzunluğu (kelime): ort. / medyan / min-maks	180 / 180 / 51-336
Özet uzunluğu (MiniLM WordPiece token): ort. / medyan	264 / 261
256 token sınırında kırpılan özet oranı	%53
Sorgu (başlık) uzunluğu: ort. kelime	10.1
Dense temsil	all-MiniLM-L6-v2, 384 boyut, float32 (üretim 305 s, benchmark dışı)
Sparse temsil	TF-IDF, sözlük 25,528, doküman başına ort. 94 sıfırdan farklı, yoğunluk %0.37
Birincil kategori (ilk 3)	cs.LG (5,967), cs.CL (3,683), cs.AI (2,692)

Örnek doküman (arXiv:2609.38923, 2026-09-30, cs.CL): *GraphForge: Training Working Agents with Graph-Anchored Workspace Synthesis* - "Working agents need to read diverse files, coordinate tools, and produce deliverables. Training such agents requires tasks built on many real files with verifiable results, but few pipelines exist to synthesize this kind of data. Existing pipelines either gene..." **Örnek sorgular:** "Piecing Together Cross-Document Coreference Resolution Datasets: Systematic Dataset Analysis and Unification"; "DiRe-RAPIDS: Topology-faithful dimensionality reduction at scale"; "Lita: Light Agent Uncovers the Agentic Coding Capabilities of LLMs".

1.2 Donanım ve yazılım

Tüm ölçümler aynı laptopta yapıldı: **Intel Core Ultra 7 255H** (16 çekirdek: 6 performans + 8 verimlilik + 2 düşük güç, hyper-threading yok), 31.4 GB RAM, GPU yok, Windows 11. Python 3.14.3, NumPy 2.4.4 (OpenBLAS), SciPy 1.17.1, scikit-learn 1.8.0, Numba 0.65.1, Dask 2026.8.0, Ray 2.59.0. **Benchmark kuralı:** her ölçüm warmup + en az 5 tekrar, raporlanan değer **medyan**. 50 ms'den kısa işlerde tekrar sayısı 25'e çıkarıldı (bkz. Bölüm 4.1). İşlemcinin hibrit yapısı sonuçları belirgin biçimde etkiledi; bu raporun ana bulgularından biri.

1.3 Deney düzeni: standart iş yükü

Fazlar arası karşılaştırmanın anlamlı olması için tek bir **standart iş yükü** tanımladık: **100 sorgu x 19,671 doküman** (tüm korpus), top-5. Tüm ana ölçümler, baseline dahil, bu iş yüküyle yapıldı; dolayısıyla raporda aynı iş için tek bir süre vardır. Bundan sapan deneyler iki türlü: (i) boyutun kendisinin *değişken* olduğu deneyler (ölçekleme eğrileri, sorgu sayısı taraması; her eğri standart noktadan geçer) ve (ii) performans değil *kavram* gösteren küçük demolar.

Tablo 3: Her deneyin iş yükü. Alt kümeler, karıştırılmış korpusun ilk N kaydıdır (iç içe: 1K ⊂ 5K ⊂ tamamı); sorgular her yerde aynı 100 başlıktır.

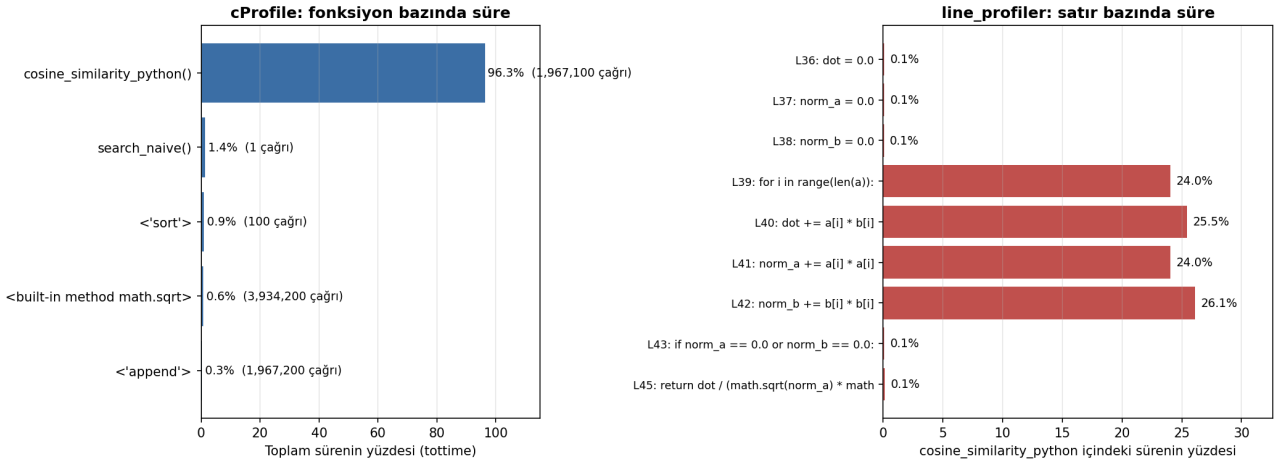
Deney	Doküman	Sorgu	Neden bu boyut
Baseline, profil (cProfile + line_profiler)	19,671	100	Standart
Sparse vs dense (3.1)	1K, 2.5K, 5K, 10K, 19.671K	100	Değişken: bellek ve süre N ile nasıl büyür
SVD karşılaştırması, recall (3.2)	19,671	100	Standart (N taraması ayrıca 1K..tamamı)
Vektörizasyon (3.3)	1K, 5K, tamamı	1, 10, 100	Değişken: iki tarama standart noktada kesişir (GEMV -> GEMM)
Gram matrisi X^TX (3.3)	2,000 x 128	-	Starter'daki ikinci işlem; saf Python üçlü döngü tam boyutta ~10 dk/ölçüm
Paralellik, online (4)	19,671	100	Standart
Paralellik, toplu (4)	19,671	2,000	100 sorgu ~ms: paralellik yükü işi aşar, ölçeklenme görünmez
GIL ve pickle demoları (4)	2,000 / 2,000	64 / 16	Kavram gösterimi (oran önemli); saf Python standart işte ~1 dk/ölçüm
Ölçekleme (6)	1K, 2.5K, 5K, 10K, 19.671K	100	Değişken (≥4 nokta, geometrik)
Doğruluk, tüm sürümler (6)	19,671	100	Standart
ANN, roofline (Ek)	19,671	100 (roofline: 1, 100, 2.000)	Standart; roofline'da sorgu sayısı değişken

2. Baseline ve Profil

Baseline, hocanın iskeletindeki gibi tamamen saf Python'dur: embedding'ler Python listesine çevrilir, her sorgu x her doküman için eleman eleman cosine hesaplanır, tüm skorlar sıralanır:

```
def cosine_similarity_python(a, b):
    dot = norm_a = norm_b = 0.0
    for i in range(len(a)):            # 384 adımlık iç döngü
        dot    += a[i] * b[i]
        norm_a += a[i] * a[i]          # sorgu normu her dokümanda YENİDEN
        norm_b += b[i] * b[i]          # doküman normu her sorguda YENİDEN
    return dot / (math.sqrt(norm_a) * math.sqrt(norm_b))
# search_naive: her sorgu için N skor -> scores.sort(key=lambda x: x[1]) -> ilk 5
```

Baseline süresi: 100 sorgu x 19,671 doküman x 384 boyut = **47.75 s** (medyan, 1 warmup + 5 ölçüm), yani sorgu başına **478 ms**: 10 ms'lik etkileşimli hedefin ~48 katı.



Şekil 1: cProfile (sol) zamanın %96'inin `cosine_similarity_python` içinde geçtiğini, line_profiler (sağ) ise bunun %100'inin dört döngü satırına neredeyse eşit dağıldığını gösteriyor: darboğaz tek bir 'pahalı satır' değil, Python yorumlayıcısının eleman başına ödediği sabit yük. Sıralama toplamın yalnızca %0.9'i.

Belirlenen üç darboğaz:

- **(1) Yorumlayıcı yükü (hesap):** 755 milyon çarp-topla adımının her biri ~10 bytecode, yeni bir float nesnesi ve liste indekslemesi demek. Eleman başına ~21 ns; CPU'nun SIMD birimleri hiç kullanılmıyor. Çözüm: vektörizasyon / JIT (Bölüm 3.3).
- **(2) Gereksiz iş:** norm satırları döngü süresinin %50'ini alıyor; ama normlar sorgudan bağımsız, indekslemede bir kez hesaplanabilir. Vektörler normalize edilince cosine = nokta çarpım olur ve arama tek bir matris çarpımına ($Q @ C^T$) indirgenir.
- **(3) Veri erişim deseni ve sıralama:** Python listesi ardışık float'lar değil, heap'e dağılmış nesnelere pointer dizisidir (önbellek dostu değil). k=5 için N elemanın tamamını O(N log N) sıralamak ve her karşılaştırmada lambda çağırmak gereksiz; O(N) seçim (argpartition) yeterli.

Saf Python içinde (2) ve (3)'ü düzelten sürüm (önceden hesaplanmış normlar, zip, heapq.nlargest) yalnızca **2.2×** kazandırdı (22.08 s). Asıl kazanç ancak (1) çözülünce gelecekti.

3. Matris İşlemleri ve Boyut İndirgeme

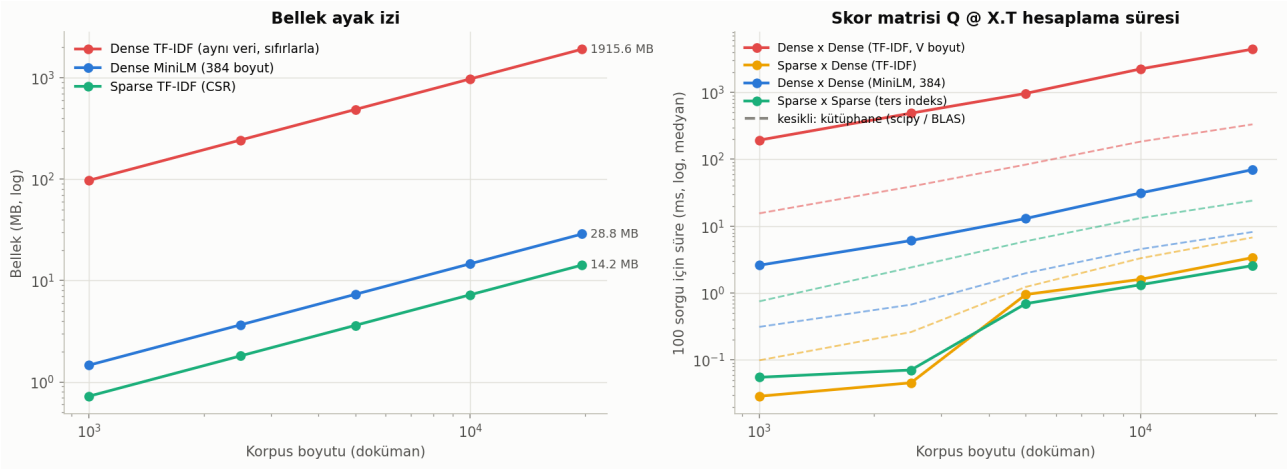
Phase 2 ölçümlerinde BLAS ve Numba thread sayısı **8'e sabitlendi**. Gerekçe Bölüm 4.1'deki thread taramasıdır: bu işlemcide OpenBLAS'ın varsayılan 16 thread'i küçük işlerde 8 thread'den kat kat yavaş.

3.1 Sparse ve dense: temsil kararı

$S = Q @ X^T$ skor matrisini dört kombinasyonda **kendi Numba çekirdeklerimizle** hesapladık ve her birini kütüphane (scipy / NumPy-BLAS) sonucuyla doğruladık (hepsi 1e-5 içinde). Sparse x sparse için arama motorlarının **ters indeks** fikrini uyguladık: X'i CSC'ye çevirince her kelimenin 'posting list'i ardışık durur; sorgunun her kelimesi için yalnızca o kelimeyi içeren dokümanlar gezilir, sıfırlara hiç dokunulmaz:

```
@njit(parallel=True)
def _ss_kernel(q_indptr, q_indices, q_data, x_indptr, x_indices, x_data, n_docs):
    out = np.zeros((nq, n_docs), np.float32)
    for i in prange(nq):                                  # her sorgu ayrı thread
        for p in range(q_indptr[i], q_indptr[i + 1]):     # sorgunun kelimeleri
            t, w = q_indices[p], q_data[p]
            for r in range(x_indptr[t], x_indptr[t + 1]): # t'nin posting listesi
                out[i, x_indices[r]] += w * x_data[r]
```

Sparse x dense çekirdeğinde korpusu (V x N) **kelime-öncelikli** sakladık: X[:, t] sütun erişimi V adımlı olur ve her eleman ayrı bir 64 B önbellek satırı çeker; $X^T[t]$ ise ardışık ve SIMD'e uygundur. Dense x dense, her iki matrisi de satır-öncelikli okuyan klasik üçlü döngüdür.



Şekil 2: Aynı TF-IDF verisi CSR'da 14.2 MB, sıfırlarıyla dense saklandığında 1916 MB (135× fazla) yer kaplıyor; MiniLM (28.8 MB) ikisinin arasında. Hesapta da sıralama aynı: %0.37 yoğunluktaki veride sparse çekirdekler 19,671 dokümanda ~2.57 ms, dense TF-IDF ise 4.44 s (sıfırları çarpmak da bellek bant genişliği ve zaman harcar). Kesikli çizgiler kütüphane sürümleri: BLAS dense işlerde bizim döngümüzden 9× hızlı, scipy sparse x sparse ise bizim ters indeksimizden 9× yavaş.

Tablo 4: Phase 2A ham sonuçları (100 sorgu, medyan). Boyutlar (1,000, 2,500, 5,000, 10,000, 19,671) eldeki toplam veriye göre ~2× geometrik aralıkla seçildi: log-log grafikte eşit aralıklı noktalar eğimi (ölçekleme üssünü) doğrudan okutur.

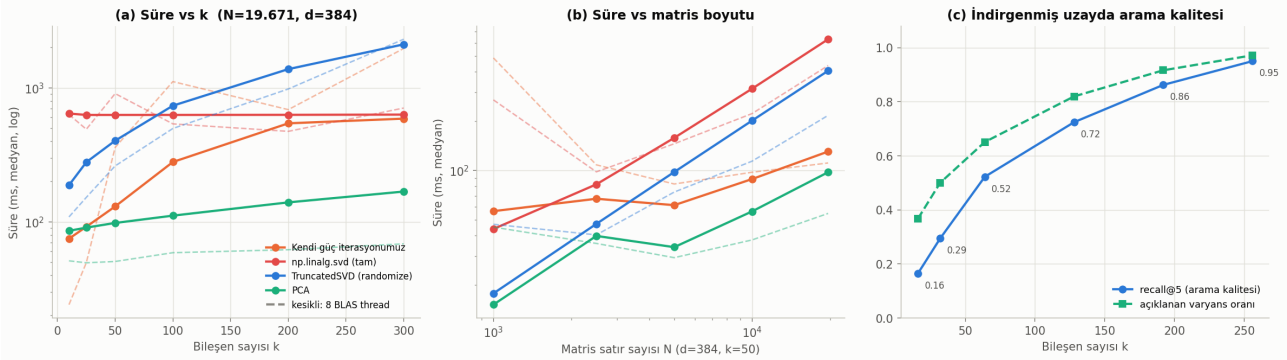
N	CSR	Dense TF-IDF	MiniLM	SS kendi / scipy	SD kendi / scipy	DDt kendi / BLAS	DDe kendi / BLAS
1,000	0.7 MB	97 MB	1.5 MB	55.1 µs / 752.9 µs	28.7 µs / 98.9 µs	194 ms / 16 ms	2.61 ms / 312.5 µs
2,500	1.8 MB	243 MB	3.7 MB	70.8 µs / 2.42 ms	45.6 µs / 261.3 µs	490 ms / 39 ms	6.10 ms / 671.9 µs
5,000	3.6 MB	487 MB	7.3 MB	695.3 µs / 5.96 ms	952.3 µs / 1.24 ms	968 ms / 83 ms	13.04 ms / 1.98 ms
10,000	7.2 MB	974 MB	14.6 MB	1.33 ms / 13.24 ms	1.61 ms / 3.32 ms	2.24 s / 184 ms	31.38 ms / 4.55 ms
19,671	14.2 MB	1916 MB	28.8 MB	2.57 ms / 24.12 ms	3.39 ms / 6.79 ms	4.44 s / 333 ms	70.05 ms / 8.20 ms

Hangisi ne zaman? Sözcük örtüşmesi aranıyorsa ve sorgu birkaç kelimeyse sparse (ters indeks) rakipsizdir: iş miktarı N'ye değil posting listelerinin uzunluğuna bağlıdır. Dense TF-IDF hiçbir durumda doğru seçim değildir. Biz final sistem için **dense MiniLM**'i seçtik: bellek olarak CSR'ın yalnızca ~2 katı, ama anlamsal benzerlik (eş anlamlılar, parafraz) yakalıyor. Sorgularımız kısa başlıklar olduğundan (ort. 10 kelime) TF-IDF'te sorgu-doküman kelime örtüşmesi çok az; ayrıca dense arama tek bir BLAS çağrısına indirgenip donanımı tam kullanabiliyor.

3.2 Boyut indirgeme: seçim ve gerekçe

Kendi **blok güç iterasyonu** SVD'mizi uyguladık: G = X^TX (384 x 384) bir kez hesaplanır (X uzun-ince, N >> d olduğu için her iterasyonda X ile çarpmaktan çok daha ucuz), sonra V <- qr(G V) tekrarlanır. İlk sürüm 200 iterasyonda bile yakınsamadı: embedding spektrumu düz (ardışık tekil değerler çok yakın) ve yakınsama hızı σ_{k+1}/σ_k oranına bağlı. İki ekleme sorunu çözdü: **oversampling** (k+10 vektörle iterasyon; oran σ_{k+11}/σ_k'ya iyileşir) ve **Rayleigh-Ritz** adımı (küçük (k+p)x(k+p) V^TGV probleminin özvektörleriyle V'yi alt uzay içinde döndürmek). Sonuç: k=50'de 32 iterasyon, tekil değerlerde max göreli hata 8.8e-07, yeniden oluşturma hatası np.linalg.svd ile aynı (76.9060 = 76.9060).

```
G = X.T @ X                                  # tek seferlik BLAS-3
V, _ = np.linalg.qr(rng.standard_normal((d, k + p)))
for it in range(max_iter):
    V, _ = np.linalg.qr(G @ V)               # güç adımı + ortonormalleştirme
    w, E = np.linalg.eigh(V.T @ (G @ V))     # Rayleigh-Ritz (küçük problem)
    V = V @ E[:, ::-1]                       # alt uzayda en iyi yönler
    if max(|Δσ²| / σ²) < tol: break
S = norm(X @ V, axis=0);  U = X @ V / S
```



Şekil 3: (a) Tam SVD k'dan bağımsız (~627 ms): 384 tekil üçlünün hepsini hesaplıyor. Kendi güç iterasyonumuz G = X^TX'i bir kez kurup sonra N'den bağımsız çalıştığı için k=10..300 aralığının tamamında tam SVD'den hızlı (k=10: 75 ms), ama maliyeti k ile büyüyor. PCA bu uzun-ince matriste kovaryans (d x d) yolunu kullandığı için k'dan neredeyse bağımsız ve k≥25'de en hızlısı. Kesikli çizgiler (8 thread) daha fazla thread'in otomatik kazanç olmadığını gösteriyor: küçük 384xk matris işlemlerinden oluşan güç iterasyonu k≥50'de 8 thread'de 1.3-4.0× yavaşlıyor (thread senkronizasyon yükü işi aşıyor). (b) N 20 kat artarken tam SVD ve TruncatedSVD ~doğrusal büyüyor, Gram yaklaşımımız (O(N d^2) bir kez + N'den bağımsız iterasyonlar) en yavaş büyüyen. (c) Asıl karar ölçütü: 384->128 indirgeme top-5 sonuçların yalnızca %72'ini koruyor; %95 için k≈256 gerekiyor.

Tablo 5: Süre (medyan), Frobenius yeniden oluşturma hatası $||X - U_k S_k V_k^T||_F$ ($||X||_F$ = 140.3) ve tepe bellek (tracemalloc), N=19,671 x 384 (embedding'ler birim normlu: $||X||_F^2 = N$).

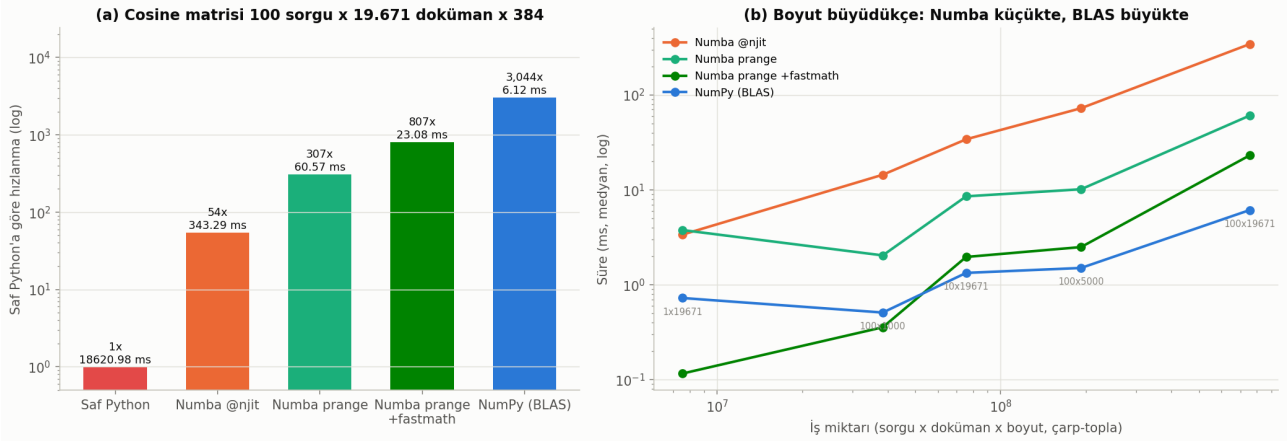
Yöntem (1 thread)	k=50: süre / hata / bellek	k=100: süre / hata / bellek	k=200: süre / hata / bellek
Kendi güç iterasyonumuz	130 ms / 76.91 / 20 MB	279 ms / 59.34 / 35 MB	543 ms / 32.99 / 66 MB
np.linalg.svd (tam)	627 ms / 76.91 / 59 MB	627 ms / 59.34 / 59 MB	628 ms / 32.99 / 59 MB
TruncatedSVD	403 ms / 77.01 / 74 MB	735 ms / 59.50 / 90 MB	1.38 s / 33.11 / 120 MB
PCA	98 ms / 76.76 / 9 MB	111 ms / 59.26 / 17 MB	140 ms / 32.95 / 32 MB

Tartışma soruları. *Truncated SVD neden tam SVD'den hızlı?* Tam SVD tüm 384 tekil üçlüyü O(N d²) maliyetle hesaplar; randomize Truncated SVD rastgele bir k+p boyutlu izdüşümle yalnızca baskın alt uzayı yakalar (O(N d k) + birkaç güç iterasyonu). *PCA ne zaman SVD'den farklıdır?* PCA veriyi önce ortalar; ortalaması sıfırdan uzak verilerde ilk SVD bileşeni 'ortalama yönü' olur ve PCA ile ayrışırlar (tablodaki PCA hatasının biraz düşük olması ortalamanın ayrıca saklanmasından gelir). Cosine araması orijine göre açıyı kullandığından biz ortalamayı çıkarmayan TruncatedSVD'yi tercih ederiz. *Truncated SVD'nin kazandığı pratik maksimum k?* Ölçümlerimizde (1 thread) k=50'de hâlâ önde, k≥100'de tam SVD'ye kaybediyor: d=384 gibi küçük bir boyutta k, d'nin ~%15-25'ini geçince rastgele izdüşüm ve güç iterasyonlarının (O(N d k)) maliyeti tam SVD'ye yetişiyor; 'kısmi' olmanın avantajı kalmıyor.

Karar: İndeksleme yöntemi olarak TruncatedSVD (ortalamasız, yani cosine geometrisini koruyor; k=256'da tam SVD ile aynı kalite). Bu boyutta kendi güç iterasyonumuz daha hızlı olurdu, ama indeksleme tek seferlik (< 1 s) bir iş olduğundan seçim sorgu süresini etkilemiyor. Ancak final sistemde indirgemeyi **varsayılan yapmadık**: bizim boyutumuzda d=384 zaten küçük; 256'ya inmek aramayı en fazla 1,5× ucuzlatırken recall@5'i %95'e düşürüyor. İndirgeme, sistemde opsiyonel bir mod olarak duruyor (`--svd 256`) ve 10x büyük veride (bellek bant genişliği sınırı) anlamlı hale geliyor (Bölüm 6).

3.3 Vektörizasyon, eleman-eleman ve JIT

Aynı işlemi (cosine matrisi $S = Q_n C_n^T$) dört (+1) yolla yazdık: saf Python döngüleri, NumPy (`Q @ C.T`, BLAS sgemm), Numba `@njit` (aynı döngü, derlenmiş), Numba `parallel=True` + `prange` ve ek olarak `fastmath=True`. Tümü NumPy ile 1e-5 içinde aynı sonucu verdi.



Şekil 4: (a) 100x19,671 cosine matrisinde saf Python 18.62 s; Numba seri aynı döngüyü yalnızca derleyerek 54×, prange 307×, fastmath ile 807×, NumPy/BLAS 3,044× hızlandırıyor. (b) Kim ne zaman kazanır: küçük işlerde Numba fastmath BLAS'ı yeniyor (çağrı ve thread uyandırma yükü yok, ara matris yok), büyük işte (100x19,671) BLAS 3.8× önde: önbellek bloklama ile her veriyi defalarca yeniden kullanıyor (247 GFLOP/s'e karşı 65).

Neden bu kadar fark? Numba seri, Python ile aynı algoritmayı yorumlayıcısız çalıştırır (nesne yok, tip kontrolü yok): 54×. Ama iç döngüdeki toplama kayan noktada birleşmeli olmadığından derleyici sırayı değiştirip SIMD kullanamaz; `fastmath` bu izni verir ve 8'li AVX2 kısmi toplamlarla tek çekirdekte bile büyük sıçrama yaratır. BLAS ise buna ek olarak **önbellek bloklama** yapar: Q ve C'yi L1/L2'ye sığan karolara bölüp her yüklenen veriyi defalarca kullanır, FMA komutlarıyla çarp-topla'yı tek komutta yapar. Gram matrisi X^TX (2000x128) aynı resmi verdi: Python 1.38 s, Numba 2.28 ms,

Numba-prange 550.5 µs, NumPy 301.1 µs. **Numba ne zaman NumPy'yi yener?** (i) İş küçükken (sabit çağrı/thread yükü baskın), (ii) birden fazla adım tek döngüde birleştirilebiliyorsa (fused: ara matris yazılmaz; Bölüm 4'teki top-k), (iii) BLAS'ın karşılığı olmayan özel döngülerde (ters indeks gibi). **Ne zaman kaybeder?** Saf GEMM gibi, onlarca yıllık optimize kütüphanelerin karşılığı olan büyük yoğun işlemlerde.

4. Paralellik ve Eşzamanlılık

4.1 Önce ölçüm: thread sayısı ve ölçüm hataları

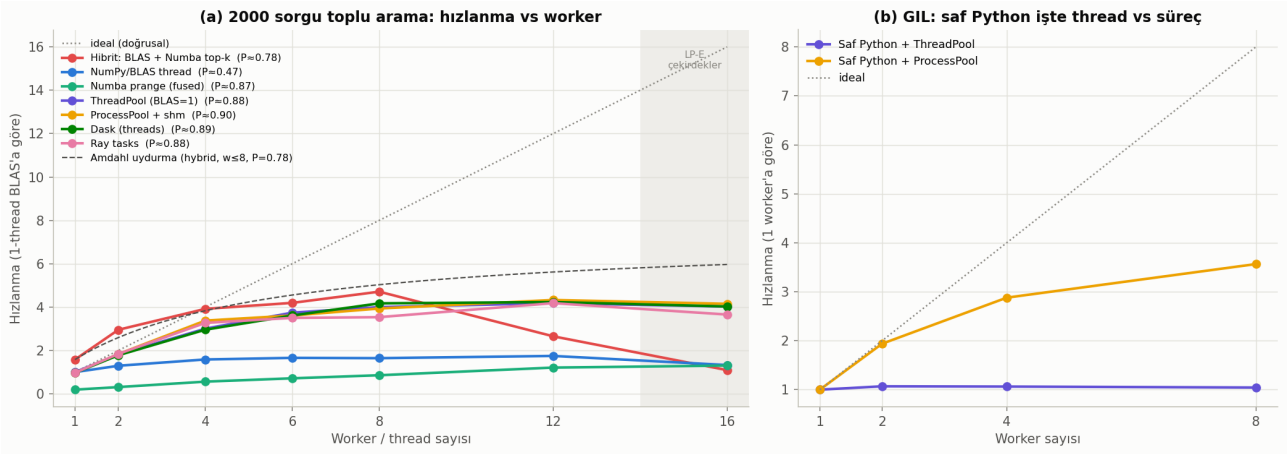
İlk bulgu bir sürprizdi: OpenBLAS varsayılan olarak 16 thread açıyor ve 100 sorguluk aramayı 8 thread'e göre belirgin biçimde YAVAŞLATIYOR (online iş yükü: 16 thread 42.08 ms, 8 thread 16.67 ms). Neden: işlemci hibrit; BLAS işi thread'lere eşit böler ve 15.-16. thread'ler düşük güç çekirdeklerine düşünce herkes en yavaş çekirdeği bekler (yük dengesizliği). Ayrıca ~1 ms'lik işlerde uyuyan thread'leri uyandırmanın gecikmesi (Windows zamanlayıcısı) işin kendisini aşıyor; aynı çağrıda 1 ms ile 19 ms arasında ölçümler gördük. Bu yüzden (i) tüm sistemi 8 thread'e sabitledik, (ii) 50 ms'den kısa işlerde tekrar sayısını 25'e çıkardık. Kendi **ölçüm hatamızı** da burada yakaladık: `threadpool_limits()` her girişte yüklü DLL'leri tarıyor (~10-20 ms); bunu zamanlanan fonksiyonun içinde çağırdığımız ilk sürüm 5 ms'lik aramayı 22 ms ölçtü. Ayar artık ölçümün dışında bir kez yapılıyor.

4.2 Stratejiler, gerekçeler ve sonuçlar

Arama işi her stratejide aynıdır (`search_block`: BLAS çarpımı + top-k); sadece dağıtma yöntemi değişir. İki iş yükü kullandık: **online** (standart iş yükü, 100 sorgu) ve **toplu** (2,000 sorgu: 'her makale için benzer makaleleri bul', öneri / yakın-kopya tespiti). 100 sorgu birkaç ms sürdüğü için paralelliğin sabit maliyetleri orada baskın; ölçeklenmeyi toplu işte görüyoruz. Sorgular worker sayısı kadar **parçaya** bölünür (sorgu başına görev değil): görev başına yük azalır ve her parça BLAS-2 (GEMV) yerine BLAS-3 (GEMM) yapar.

Tablo 6: Toplu iş yükü (2,000 sorgu x 19,671 doküman), medyan. Tüm stratejilerin top-5 sonuçları referansla 1e-5 içinde aynı. *Hızlanma: en iyi sürenin 1-thread BLAS'a göre oranı. P (Amdahl): stratejinin kendi 1-worker süresine göre, w≤8 noktalarıyla uydurulan paralel oran. Naive baseline bu iş için tahminen 955.04 s (Faz 1'in doküman-sorgu başına maliyetinden ekstrapolasyon).

Strateji	Neden denedik	1 worker	8 worker	En iyi (w)	Hızlanma*	P
NumPy/BLAS thread'leri	Örtük paralellik, sıfır kod; her zaman ilk adım	439 ms	267 ms	251 ms (12)	1.7×	0.47
Numba prange (fused)	Benzerlik+top-k tek döngüde, ara matris yok	2.30 s	513 ms	337 ms (16)	1.3×	0.87
Hibrit: BLAS + Numba top-k	GEMM BLAS'ta, seri top-k Numba'da paralel	278 ms	93 ms	93 ms (8)	4.7×	0.78
ThreadPoolExecutor	BLAS/argpartition GIL'i bırakır -> thread yeter	440 ms	110 ms	105 ms (12)	4.2×	0.88
ProcessPool + shared memory	GIL yok; korpus kopyalanmaz (shm), sadece sorgu gider	456 ms	112 ms	101 ms (12)	4.3×	0.90
Dask (delayed, threads)	Görev grafiği; out-of-core için hazır	456 ms	105 ms	104 ms (12)	4.2×	0.89
Ray tasks	Çok düğüme taşınabilir; ray.put ile sıfır kopya	454 ms	124 ms	105 ms (12)	4.2×	0.88
Ray actors	Durumlu worker (model servisi gibi)	450 ms	127 ms	117 ms (12)	3.8×	-



Şekil 5: (a) Toplu aramada saf BLAS thread'leri 1.7×'de tıkanıyor: matris çarpımı ölçekleniyor ama tek thread'li argpartition seri kalıyor (Amdahl P=0.47). Top-k'yı da parçalara bölen stratejiler (ThreadPool, ProcessPool, Dask, Ray) ~4×'e, top-k'yı Numba ile paralelleştiren hibrit 8 worker'da 4.7×'e çıkıyor. Hiçbiri ideal doğruya yaklaşmıyor: 6 P-çekirdekten sonra eklenen E-çekirdekler daha yavaş ve tüm thread'ler aynı bellek bant genişliğini paylaşıyor. Hibrit 12-16 worker'da çöküyor (402 ms): BLAS'a ve Numba'ya ayrı ayrı w thread verildiği için 2w thread 16 çekirdek için yarışıyor (aşırı abonelik; OpenBLAS thread'leri iş bitince bir süre meşgul-bekliyor). Amdahl uydurması bu yüzden w≤8 noktalarıyla yapıldı. (b) Saf Python'da thread'ler GIL yüzünden hiç hızlanmıyor, süreçler ölçekleniyor: GIL'in doğrudan kanıtı.

GIL. NumPy/BLAS ve argpartition C kodunda çalışırken GIL'i bırakır; bu yüzden ThreadPool, ProcessPool ile neredeyse aynı hızlanmayı süreç başlatma ve IPC maliyeti olmadan verir. Saf Python işte ise tablo tersine döner (Şekil 5b): 8 thread 1.0×, 8 süreç 3.6×. (Not: 1 worker'lı süreç, aynı işi yapan tek thread'den 2.4× yavaş; Windows arka plandaki çocuk süreçleri büyük olasılıkla yavaş E-çekirdeklere yerleştiriyor. Hibrit işlemcinin bir başka etkisi.) Süreçlerin bedeli: Windows'ta 'spawn' ile havuz kurulumu ~1.2 s ve veri kopyalama. Hocanın iskeletindeki 'kötü' sürüm (korpus her görevde pickle, 6.6 MB) 601.70 ms, parçalı 'iyi' sürüm 413.96 ms sürdü. NumPy sürümünde korpusu `multiprocessing.shared_memory`'ye bir kez koyduk; worker'lar onu kopyalamadan görüyor, görevlere sadece sorgu parçası gidiyor.

asyncio (I/O senaryosu): Sorgu metni önce uzak bir embedding API'sine gidiyorsa (gecikme simüle) 100 sorguyu sırayla beklemek 10 ms gecikmede 1.57 s, `asyncio.gather` ile 96.00 ms (16×); 10 eşzamanlı istekle sınırlandırınca (gerçek API rate limit'leri) 285.12 ms. Gecikme sıfırken (embedding önbellekte) kayda değer bir kazanç yok (53.00 ms vs 44.85 ms; fark ölçüm gürültüsü düzeyinde): asyncio tek thread'dir, CPU işini hızlandırmaz. Bu yüzden async'i arama çekirdeği için değil, sistemin I/O kenarı için uygun görüyoruz.

Dask ve Ray. İkisi de doğru sonuç verdi ve ThreadPool/ProcessPool düzeyinde hızlandı, ama tek makinede ve RAM'e sığan veride ek bir şey katmıyorlar: Dask görev grafiği, Ray ise 5 s'lik başlatma + nesne deposu yükü getiriyor. Online iş yükünde Ray görev başına yükü nedeniyle en iyi 13.92 ms ile hibritin (9.05 ms) gerisinde. Dask'ın gerçek yeri korpus RAM'e sığmadığında (chunk'lı out-of-core çarpım), Ray'inki birden fazla makineye çıkıldığında.

Tasarım sorusu: kod karmaşıklığı başına en iyi hızlanma? Hibrit strateji: tek satır `topk_numba(Q @ C.T, k)` ve bir thread ayarı; havuz, IPC, serileştirme yok. ProcessPool benzer hız için ~60 satır (shared memory, initializer, temizlik) ve saniyelerce başlatma istiyor. **Korpus 10× büyük olsaydı** (196,710 doküman, 384 boyut float32 = 288 MB) hâlâ RAM'e sığar ve cevabımız değişmezdi; skor matrisi (q x N) büyüdüğü için sorgu parçalama (ThreadPool) ve korpus parçalama önem kazanırdı. 100× (RAM sınırı) ölçekte Dask ile out-of-core blok çarpımı ya da ANN (Ek A) gerekirdi.

5. Tasarım Kararları

Tablo 7: Tasarım kararları, alternatifler ve dayandıkları ölçümler.

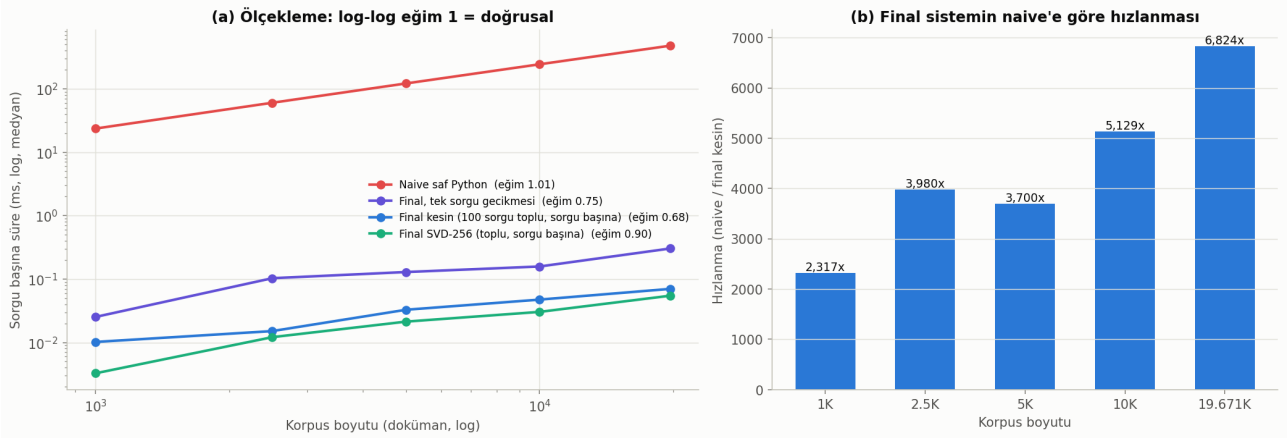
Karar	Seçim	Düşünülen alternatifler	Veri / gerekçe
Temsil	Dense MiniLM, float32	Sparse TF-IDF, dense TF-IDF	CSR 14.2 MB, MiniLM 28.8 MB, dense TF-IDF 1916 MB; MiniLM anlamsal + tek BLAS çağrısı (Şekil 2)
Normlar	İndekslemede bir kez	Her karşılaştırmada	Profilde döngünün %50'i (Şekil 1)
Benzerlik çekirdeği	BLAS sgemm (toplu)	Numba fused, saf Python	100x19,671: BLAS 6.12 ms, Numba-fm 23.08 ms (Şekil 4)
Top-k	Numba prange, O(N)	sort O(N log N), argpartition (1 thread)	argpartition BLAS hızlandıktan sonra seri darboğaz; Numba 10× hızlı (Bölüm 4)
Thread sayısı	8 (BLAS + Numba)	Varsayılan 16, 1, 12	Online: 16 thr 42.08 ms, 8 thr 16.67 ms; hibrit CPU yük dengesizliği
Paralellik	Hibrit (BLAS + Numba)	ThreadPool, ProcessPool, Dask, Ray, asyncio	Toplu: hibrit 93.20 ms, en iyi süreç 101.46 ms; en az kod
Boyut indirgeme	Opsiyonel (TruncatedSVD-256)	Güç iterasyonu, PCA, tam SVD; k=128	k=256 recall 95%, k=128 recall 72%; kazanç ≤1.5× (Şekil 3)
Sorgu toplama	Toplu (batch) arama	Sorgu başına GEMV	Roofline: q=1 bant genişliği sınırlı, q=100 hesap sınırlı (Ek B)

6. Final Sistem ve Ölçekleme

Hat: metin -> MiniLM embedding -> (opsiyonel TruncatedSVD) -> L2 normalize (indekslemede, bir kez) -> `Q @ C.T` (BLAS, 8 thread) -> `topk_numba` (prange, 8 thread) -> top-5. Çok sorguda sorgular 512'lik bloklar halinde, tek bir yeniden kullanılan skor tamponuyla işlenir (her çağrıda yüzlerce MB yeni bellek ayırıp Windows'ta sayfa hatası ödememek için). Çekirdeğin özü:

```
def search_hybrid(Qn, Cn, k, block=512):
    if len(Qn) <= block:
        return topk_numba(Qn @ Cn.T, k)          # BLAS sgemm + paralel O(N) top-k
    buf = np.empty((block, len(Cn)), np.float32) # bir kez ayrılan skor tamponu
    for a in range(0, len(Qn), block):
        S = buf[:min(block, len(Qn) - a)]
        np.matmul(Qn[a:a + len(S)], Cn.T, out=S) # yeni bellek ayırmadan yaz
        out_idx[a:a + len(S)], out_sc[a:a + len(S)] = topk_numba(S, k)

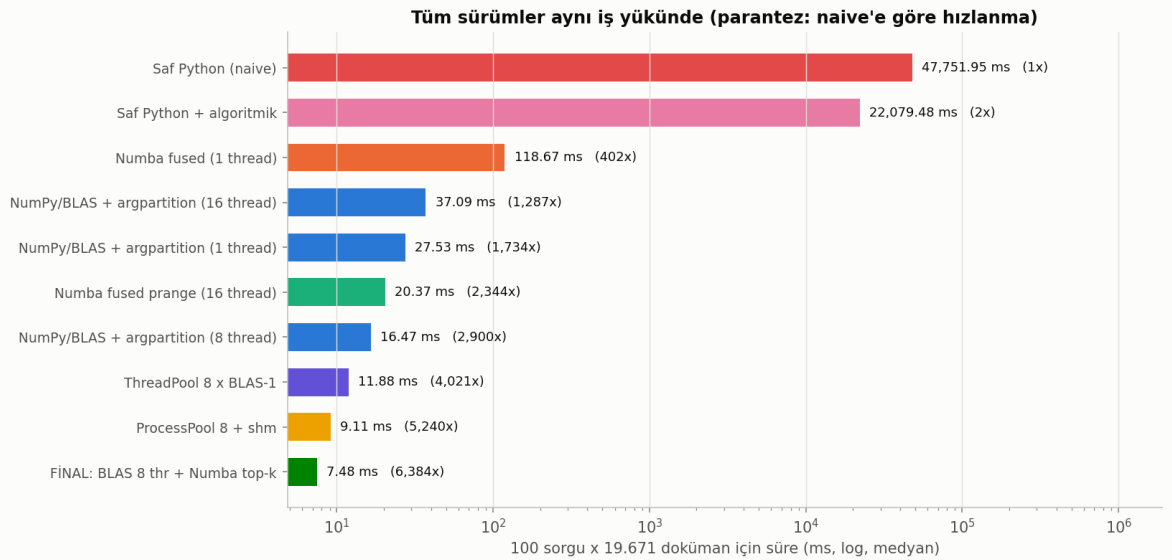
class SimilaritySearchEngine:                    # indeksleme: SVD (ops.) + normalize BİR KEZ
    def search_prepared(self, Qn, k=5):
        return search_hybrid(Qn, self.C, k)
```



Şekil 6: (a) Naive baseline log-log eğim 1.01 ile tam doğrusal büyüyor (O(N d)). Final sistemin tek sorgu gecikmesi son aralıkta (10,000 -> 19,671) eğim 0.96 ile doğrusal: korpus L3'ten taşınca her sorgu tüm korpusu RAM'den bir kez okuyor (bant genişliği sınırlı, Ek B). Toplu modda eğim tüm aralıkta 0.68, son aralıkta 0.58: iş hâlâ O(N d), ama ~ms'lik toplu işte süreyi kısmen N'den bağımsız sabit maliyetler (çağrı, thread uyandırma) belirliyor; N büyüdükçe bunlar amorti olduğu için alt-doğrusal görünüyor. 19,671 dokümanda naive sorgu başına 478 ms, final 70.0 µs (100 sorgu toplu), tek sorgu gecikmesi 303.0 µs: etkileşimli hedefin çok altında. (b) Hızlanma N ile artıyor çünkü büyük işte sabit yükler amorti ediliyor.

Tablo 8: Doğruluk kontrolü. Baseline float64 saf Python, final float32 BLAS; fark float32 yuvarlamasından (~1e-7). Eşit skorlu dokümanlarda sıra değişimi hata sayılmaz (tie-aware kontrol).

Kontrol	Sonuç
Kesin mod vs naive baseline (tüm korpus, 100 sorgu)	GEÇTİ · max |skor farkı| = 2.9e-07 (tolerans 1e-5) · indeks sırası farklı sorgu: 0
SVD-256 modu vs aynı uzayda naive	GEÇTİ · max fark 6.9e-07 · kesin moda göre recall@5 = 0.950
Phase 2-3'teki tüm uygulamalar	Her biri referansla 1e-5 içinde (results/*.json içinde 'correct')



Şekil 7: Aynı iş yükünde (100 sorgu x 19,671 doküman) hikâyenin tamamı: saf Python 47.75 s'den final sistemin 7.48 ms'sine 6,384×. Kazancın büyük kısmı tek bir adımdan, yorumlayıcıyı BLAS/JIT ile devre dışı bırakmaktan geliyor (tek thread BLAS: 1,734×); sonraki her adım (thread ayarı, top-k paralelleştirme) birkaç kat ekliyor. 16 thread'li varsayılan BLAS tek thread'den yavaş. Numba fused (16 thread, 20.37 ms) ara matris yazmasa da önbellek bloklaması olmadığı için finalin 2.7× gerisinde; ProcessPool (9.11 ms) yakın ama ~60 satır ek kod ve saniyelerce havuz kurulumu istiyor.

Doğrusal ölçekleniyor mu? İş miktarı doğrusal (O(N d)) ve hiçbir yerde doğrusalın üstüne çıkmıyoruz: tek sorgu gecikmesi büyük N'de doğrusal (son aralıkta eğim 0.96), toplu modda ise sabit maliyetler henüz tam amorti olmadığı için alt-doğrusal (eğim 0.58). 19,671 dokümanda korpus 29 MB, L3 önbelleği (24 MB) aşınca tek sorgu bellek bant genişliğiyle sınırlanır (Ek B). **10× veriyle** (~197K doküman, ~288 MB) doğrusal tahmin tek sorgu için ~3.0 ms, hâlâ etkileşimli. Daha büyük ölçekte sırasıyla: float16/int8 niceleme (bant genişliğini 2-4× azaltır), SVD-256 (1,5×) ve sonunda ANN (IVF; Ek A'da kendi uygulamamızla aday kümesini korpusun ~%6'sına indirip recall@5≈0.90 elde ettik) yapardık.

7. Tartışma ve Dersler

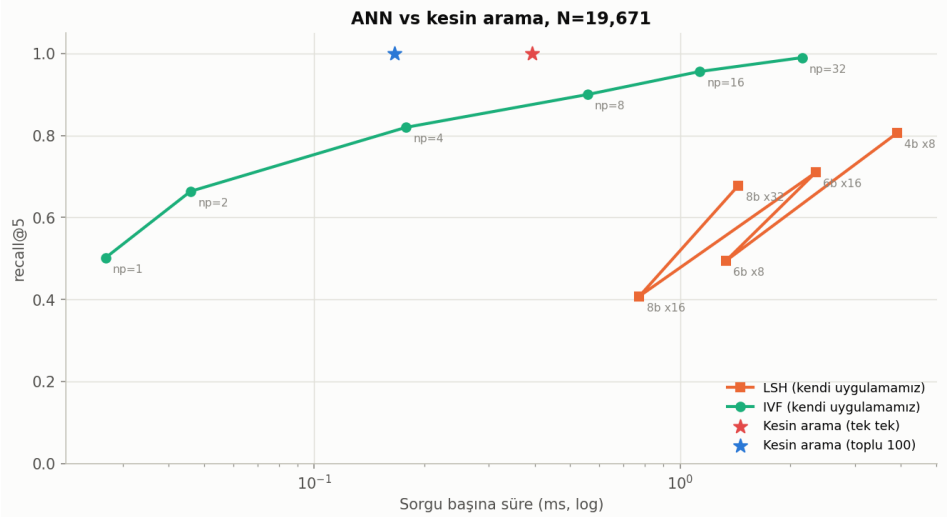
- **En büyük sürpriz: daha çok thread daha yavaş olabilir.** Hibrit işlemcide varsayılan 16 BLAS thread'i, 8 thread'den kat kat yavaştı; küçük SVD matrislerinde 1 thread 8 thread'den hızlıydı. Amdahl'ın 'seri kısım' kavramı burada thread yönetimi ve en yavaş çekirdeği bekleme olarak çıktı.
- **Amdahl'ı canlı gördük:** BLAS'ı hızlandırdıkça, profilde %1 bile görünmeyen top-k toplam sürenin çoğuna dönüştü. Darboğaz yer değiştirir; her optimizasyondan sonra yeniden profil almak gerekir.
- **Ölçüm de bir sistemdir:** threadpoolctl'in giriş maliyeti ve Windows'un thread uyandırma gürültüsü sonuçları 4-10× bozabiliyordu. Tek thread'li saf Python baseline bile iki ayrı çalıştırmada 30,8 s ve 11,9 s ölçüldü (aynı kod ve veri; işletim sistemi işi P- ya da E-çekirdeğe yerleştirdi). Bu yüzden her karşılaştırmayı aynı çalıştırmanın içinden yaptık; medyan, yeterli tekrar ve ayarları ölçüm dışında yapmak şart.
- **GIL ancak saf Python'da sorun:** İş C'ye (BLAS, argpartition) devredildiğinde thread'ler süreçler kadar iyi; süreçlerin başlatma/IPC maliyetine yalnızca saf Python işte değer.
- **Farklı yapardık:** sorguları baştan toplu (batch) düşünmek; Numba'yı benzerlik için değil, BLAS'ın olmadığı yerde (top-k, ters indeks) kullanmak; bir de MiniLM'in 256 token sınırı yüzünden özetlerin

%53'inin kırpıldığını baştan bilip daha uzun bağlamlı bir model seçmek.

- **Gerçek AI sistemleriyle bağlantı:** RAG hatlarındaki vektör veritabanları (FAISS, pgvector, Milvus) tam olarak bu adımları yapar: normalize embedding + BLAS'la toplu iç çarpım + paralel top-k; ölçek büyüyünce IVF/HNSW ve niceleme. LLM servislerindeki 'batching' de roofline'daki aynı mantıkla bellek sınırlı işi hesap sınırlı hale getirir.

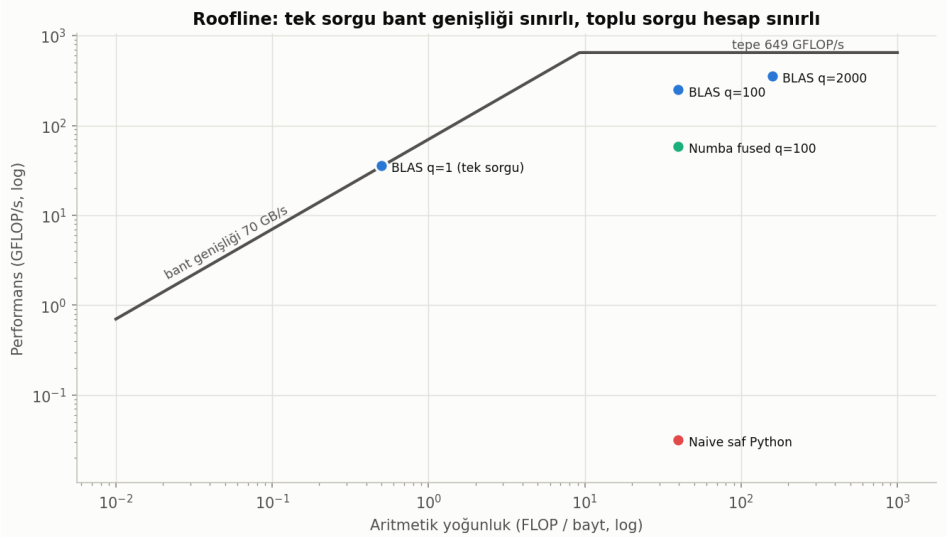
Ek

Ek A (bonus): Yaklaşık en yakın komşu (kendi LSH ve IVF uygulamamız)



Şekil 8: N=19,671: IVF (n_list=140, nprobe=8) korpusun %6'ine bakarak recall@5=0.90, sorgu başına 0.56 ms. Kesin tek sorgu 0.39 ms, toplu kesin arama sorgu başına 0.165 ms. IVF ancak recall≈0.82'ye razı olunca (nprobe=4) kesin tek sorgudan hızlı; Python'daki sorgu başına döngü yükü kazancı yiyor. LSH'nin recall 0.81'ye ulaşması için korpusun %40'ine bakması gerekiyor: benzer makalelerin cosine'i ~0.6 (açı ~53°), bu açıda bir bitte çarpışma olasılığı ~0.70, b bitte 0.70^b; yüksek recall için az bit = büyük kovalar gerekiyor. Sonuç: bu ölçekte toplu kesin arama ANN'den hızlı ve kayıpsız; ANN (derlenmiş bir kütüphaneyle) tek sorgu gecikmesinde ve çok daha büyük N'de anlam kazanır.

Ek B (bonus): Roofline analizi



Şekil 9: Ölçülen tepe 649 GFLOP/s (sgemm) ve bant genişliği 70 GB/s; kırılma noktası 9 FLOP/bayt. Tek sorgu araması AI≈0.5 ile bant genişliği sınırlı bölgede (36 GFLOP/s, RAM tavanının %102'i: tavanın kendisinde. Tavan 1 GB'lık diziyle, yani doğrudan RAM'den ölçüldü; 29 MB'lık korpusun bir kısmı tekrarlanan ölçümlerde 24 MB'lık L3'te kaldığı için ~%100 civarı, hatta biraz üstü görülebiliyor), 100 sorgu toplu arama AI≈40 ile hesap sınırlı bölgede (250 GFLOP/s, tavanın %38'i). Sorguları toplamak her dokümanı bellekten bir kez okuyup 100 kez kullanmak demek: batching'in roofline açıklaması.

Ek C: Tekrar üretme ve ham veri

Kod: https://github.com/sevvalsaritas/hpp-for-ai-project. Kurulum ve tüm sonuçları yeniden üretme:

```
py -3.14 -m pip install -r requirements.txt
py -3.14 run_all.py             # veri (önbellekli) -> tüm benchmark'lar -> figürler -> bu PDF
py -3.14 demo.py --compare      # canlı demo
```

Ham sonuçlar (her ölçümün tüm tekrarları dahil) `results/` altında: dataset_stats.json, phase1_baseline.json, profile_cprofile.txt, profile_line.txt, phase2a_sparse_dense.csv, phase2b_svd_vs_k.csv, phase2b_svd_vs_n.csv, phase2c_vectorise.csv, phase3_parallel.csv,

phase3_gil_demo.csv, phase3_async.csv, phase4_scaling.csv, phase4_all_versions.csv, bonus_ann.csv, bonus_roofline.json, hardware.json.

Tablo 9: Ölçekleme testi ham verisi (Şekil 6), medyan.

N	Naive / sorgu	Final / sorgu (toplu)	Final tek sorgu	SVD-256 / sorgu	Hızlanma
1,000	23.6 ms	10.2 µs	25.3 µs	3.3 µs	2,317×
2,500	60.2 ms	15.1 µs	103.1 µs	12.1 µs	3,980×
5,000	121.3 ms	32.8 µs	129.0 µs	21.3 µs	3,700×
10,000	243.2 ms	47.4 µs	157.8 µs	30.4 µs	5,129×
19,671	477.5 ms	70.0 µs	303.0 µs	54.6 µs	6,824×

Tablo 10: Tüm sürümler (Şekil 7) ham verisi. Final sistem aynı iş yükünde Şekil 6'nın son noktasında bağımsız bir ölçümle 7.00 ms çıktı; aynı kodun tekrar ölçümleri arasındaki fark %7. Saf Python satırları ve Şekil 6'daki naive son nokta aynı Phase 1 ölçümüdür.

Sürüm (100 sorgu x 19,671 doküman)	Medyan	Naive'e göre
Saf Python (naive)	47.75 s	1.0×
Saf Python + algoritmik	22.08 s	2.2×
Numba fused (1 thread)	118.671 ms	402×
NumPy/BLAS + argpartition (16 thread)	37.090 ms	1,287×
NumPy/BLAS + argpartition (1 thread)	27.533 ms	1,734×
Numba fused prange (16 thread)	20.370 ms	2,344×
NumPy/BLAS + argpartition (8 thread)	16.467 ms	2,900×
ThreadPool 8 x BLAS-1	11.876 ms	4,021×
ProcessPool 8 + shm	9.112 ms	5,240×
FİNAL: BLAS 8 thr + Numba top-k	7.480 ms	6,384×

Bağımlılıklar (requirements.txt): numpy, scipy, scikit-learn, matplotlib, sentence-transformers, torch, transformers, line_profiler, numba, threadpoolctl, dask, ray, psutil, reportlab.

YZ kullanımı: Kod ve rapor taslağı bir YZ asistanı yardımıyla yazıldı; tüm deneyler bu makinede koşuldu, her figür ve kod bloğu tarafımızdan incelendi ve açıklanabilir durumda.